

Chapter 4:
Logic gates and logic circuits

Learning objectives

By the end of this chapter you should be able to:

- use logic gate symbols for NOT, AND, OR, NAND, NOR and XOR
- understand and define the functions of the NOT, AND, OR, NAND, NOR and XOR (EOR) gates
- construct the truth table for each of the above logic gates
- construct a logic circuit from:
  - a problem statement
  - a logic expression
  - a truth table
- construct a truth table from:
  - a problem statement
  - a logic circuit
  - a logic expression
- construct a logic expression from:
  - a problem statement
  - a logic circuit
  - a truth table.

4.01 Boolean logic and problem statements

Consider the following question:

Is Colombo further north than Singapore?

In everyday language the answer will be either yes or no. ('Yes', in fact.) However, the question could be rephrased to make use of the language of Boolean logic:

Colombo is further north than Singapore: TRUE or FALSE?

More formally, the statement:

Colombo is further north than Singapore.

can be described as an example of a logic assertion or a **logic proposition** that can have only one of the two alternative Boolean logic values: TRUE or FALSE.

KEY TERM

Logic proposition: a statement that is either TRUE or FALSE

Now consider the following two individual statements.

- You should take an umbrella if *it is raining* or if *the weather forecast is for rain later.*
- The air-conditioning system is set to come on in an office only *during working hours* but also only *if the temperature rises to above 25°C.*

Each of these statements contains two logic propositions which are highlighted. In each statement these logic propositions are combined in some way. Finally, each statement has the addition of an outcome which is dependent on the combination of the two propositions. Each of these is, therefore, an individual example of a **problem statement**.

KEY TERM

Problem statement: an informal definition of an outcome which is dependent on one logic proposition or a combination of two or more logic propositions

4.02 Boolean operators

The problem statements identified above can be more formally expressed in a form that is suitable for handling with Boolean logic. To do this it is necessary to use Boolean operators. The three basic Boolean operators are AND, OR and NOT.

The definition for AND, OR and NOT can be expressed as:

- A AND B is TRUE if A is TRUE and B is TRUE
- A OR B is TRUE if A is TRUE or B is TRUE
- NOT A is TRUE if A is FALSE.

Here, both A and B represent any logic proposition or assertion that has a value TRUE or FALSE.

The two problem statements above can now be rephrased as follows:

- Take_umbrella = TRUE IF (raining = TRUE) OR (rain_forecast = TRUE)
- System_on = TRUE IF (office hours = TRUE) AND (temperature > 25°C).

Each original problem statement has now been rephrased to include a form of **logic expression**. The format of each expression here does not follow any formally defined convention but the structure does allow the underlying logic to be understood. In general, a logic expression consists of logic propositions combined using Boolean operators. The expression may be included in an equation with a defined output.



KEY TERM

Logic expression: logic propositions combined using Boolean operators, which may be equated to a defined outcome

Any logic expression can be constructed using only the Boolean operators AND, OR and NOT but it is often convenient to use other operators. Here are the definitions for the three other operators that you should be familiar with:

- A NAND B is TRUE if A is FALSE or B is FALSE
- A NOR B is TRUE if A is FALSE and B is FALSE
- A XOR B is TRUE if A is TRUE or B is true but not both of them.

WORKED EXAMPLE 4.01

Constructing a logic expression from a problem statement

Consider the following problem statement.

A shopkeeper orders a delivery of goods at the end of each month. However, if the stock of a particular item falls to the re-order level before the end of the month, a delivery is ordered immediately. Also, if a regular customer orders a large amount of goods, a delivery is ordered immediately.

We need to identify the conditions in the statement that can have true or false values. These can be underlined:

A shopkeeper orders a delivery of goods at the end of each month. However, if the stock of a particular item falls to the re-order level before the end of the month a delivery is ordered immediately. Also, if a regular customer orders a large amount of goods a delivery is ordered immediately.

The conditions can now be collected together in one logic expression:

End_of_month OR re-order_level_reached OR (regular_customer AND large_amount)

To simplify this we change each condition into a symbol.

- Let A represent End_of_month.
- Let B represent re-order_level_reached.
- Let C represent regular customer.
- Let D represent large_amount.

The logic expression can now be written in an equation using X to represent 'a delivery is ordered':

X = A OR B OR (C AND D)

TASK 4.01

Convert the conditions in the following problem statement into a simple logic expression:

A document can only be copied if it is not covered by copyright or if there is copyright and permission has been obtained.

4.03 Truth tables

The truth table is a simple but powerful technique for representing any logic expression or for describing the possible outputs from a logic circuit.

A truth table is presented by making use of the convention that TRUE can be represented as 1 and FALSE can be represented as 0. The simplest use of a truth table is to represent the logic associated with a Boolean operator.

As an example, let us consider the AND operator. The labelling of the truth table follows the convention that the initially defined values are represented by A and B and the value obtained from the simple expression using the AND operator is represented by X. In other words, we write the truth table for X = A AND B. Remembering that AND only returns TRUE if both A and B are TRUE we expect a truth table with only one instance of X having the value 1. The truth table is shown in Table 4.01.

The truth table has four rows corresponding to the four combinations of the truth values for A and B. Three of these lead to a 0 in the X column as expected.

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

Table 4.01 The truth table for the AND operator



TIP

When constructing a truth table make sure that the left-hand columns for the input values are written as though they were increasing binary values.

TASK 4.02

Without looking further on in the chapter, construct the truth table for the OR operator.

4.04 Logic circuits and logic gates

The digital circuits that constitute the inner workings of a computer system operate as logic circuits where each individual part of the circuit is either in an 'on' state, corresponding to a 1, or in an off state, corresponding to a 0. A logic circuit comprises component parts called logic gates. Each different **logic gate** has an operation that matches a Boolean operator.

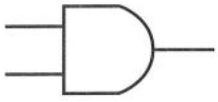


Figure 4.01 The symbol for the AND logic gate

KEY TERM

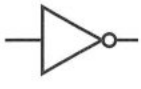
Logic gate: a component of a logic circuit that has an operation matching that of a Boolean operator

When drawing a circuit, standard symbols are used for the logic gates. As an example, the symbol shown in Figure 4.01 represents an AND gate.

The first point to note here is that the shape of the symbol tells us the type of gate. The second point is that the inputs are shown on the left-hand side and the output is shown on the right-hand side. In general, the number of inputs is not limited to two. We will only consider circuits where the number of inputs is two or fewer.

Figure 4.02 shows the logic gate symbols and the associated truth tables for each of the six Boolean operators introduced in Section 4.02.

NOT



A	X
0	1
1	0

AND



A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

OR



A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

NAND



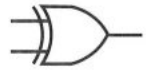
A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

NOR



A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

XOR



A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

Figure 4.02 Logic gate symbols and their associated truth tables

There are two other points to note here. The NOT gate is a special case, having only one input. The second point is that a NAND gate is a combination of a NAND gate followed by a NOT gate, and a NOR gate is a combination of an OR gate followed by a NOT gate. NAND and NOR gates produce a complementary output to the AND and OR gates.

TASK 4.03

Draw a circuit where A and B are input to an AND gate, from which the output is carried to a NOT gate, from which there is an output X. Show that this has the same outcome as having one NAND gate.

Extension Question 4.01

Could the same outcome be produced by positioning a NOT gate before the AND gate?

TIP

You need to remember the symbol for each of these gates. A good start here is to remember that AN**D** has the proper **D** symbol and OR has the curvy one.

You also need to remember the definitions for the gates so that you can construct the corresponding truth table for each gate.

Question 4.01

Can you recall from memory the symbols and definitions of the six logic gates introduced in this chapter?

WORKED EXAMPLE 4.02

Constructing a logic circuit from a problem statement or logic expression

You need to be able to construct a logic circuit from either a problem statement or from a logic expression. If you are given a problem statement, the best approach is to first convert it to a logic expression.

Consider the following problem statement: A bank offers a special lending rate to customers subject to certain conditions. To qualify, a customer must satisfy certain criteria.

- The customer has been with the bank for two years.
- Two of the following conditions must also apply:
  - the customer is married
  - the customer is aged 25 years or older
  - the customer's parents are customers of the bank.

To convert this statement to a logic expression using symbols we can choose:

- let A represent an account held for two years
- let B represent that the customer is married
- let C represent that the customer is aged 25 years or older
- let D represent that the customer's parents have an account.

(Continued)